



OPEN A hybrid super learner ensemble for phishing detection on mobile devices

Routhu Srinivasa Rao^{1,2}, Cheemaladinne Kondaiah³, Alwyn Roshan Pais³ & Bumshik Lee^{2✉}

In today's digital age, the rapid increase in online users and massive network traffic has made ensuring security more challenging. Among the various cyber threats, phishing remains one of the most significant. Phishing is a cyberattack in which attackers steal sensitive information, such as usernames, passwords, and credit card details, through fake web pages designed to mimic legitimate websites. These attacks primarily occur via emails or websites. Several antiphishing techniques, such as blacklist-based, source code analysis, and visual similarity-based methods, have been developed to counter phishing websites. However, these methods have specific limitations, including vulnerability to zero-day attacks, susceptibility to drive-by-downloads, and high detection latency. Furthermore, many of these techniques are unsuitable for mobile devices, which face additional constraints, such as limited RAM, smaller screen sizes, and lower computational power. **To address these limitations, this paper proposes a novel hybrid super learner ensemble model named Phish-Jam, a mobile application specifically designed for phishing detection on mobile devices.** Phish-Jam utilizes a super learner ensemble that combines predictions from diverse Machine Learning (ML) algorithms to classify legitimate and phishing websites. By focusing on extracting features from URLs, including handcrafted features, transformer-based text embeddings, and other Deep Learning (DL) architectures, the proposed model offers several advantages: fast computation, language independence, and robustness against accidental malware downloads. From the experimental analysis, it is observed that the super learner ensemble achieved significant accuracy of 98.93%, precision of 99.15%, MCC of 97.81% and F1 Score of 99.07%.

Keywords Phishing URLs, Super learner ensemble, Phish-Jam, Text embedding transformers, Deep learning, Machine learning

The rapid adoption of mobile devices for online activities, from communication and social networking to e-commerce and financial transactions, has made them a prime target for cyberattacks. Among these, phishing is one of the most pervasive and dangerous threats. Phishing attacks exploit users by tricking them into revealing sensitive information, such as login credentials and financial details, through fraudulent websites that mimic legitimate ones. With mobile devices increasingly being used to access the internet, detecting and mitigating phishing attacks in this domain has become a pressing concern.

In phishing, the attackers draw legitimate users by attaching cloned website links through email, Facebook, or Twitter. Attackers choose phishing because it is lucrative and easy to perform due to the existence of phishing toolkits. Because of digitization, the ease of fraud has also increased multifold. As per the 2024 Phishing Trends and Intelligence report¹, PHISH LABS found that total phishing volume rose 40.9 percent in 2023 compared to the previous year. According to Microsoft 2024 Global Tech Support Scam Research report², there has been a rise of 69% of attacks from January 2023 to September 2024. The statistics demonstrate that nearly half of the users surveyed are victims of phishing attacks. According to the Kaspersky report, India is among the top 3 countries facing phishing attacks through social media such as WhatsApp, Telegram, Viber, Twitter, etc. From the latest report from Anti-Phishing Working Group (APWG) 2024 Q3 trends³, it is observed that 1,025,968 phishing attacks were performed, which remains the highest count and the worst quarter since the inception of

¹Department of Computer Science and Engineering, Gandhi Institute of Technology and Management, Visakhapatnam, Andhra Pradesh 530045, India. ²Department of Information and Communication Engineering, Chosun University, Gwangju 61452, Republic of Korea. ³Information Security Research Lab, Department of Computer Science and Engineering, National Institute of Technology Karnataka, Mangalore, Surathkal 575025, India. ✉email: bslee@chosun.ac.kr

APWG. Also, the 2024 Q3 quarter is the third quarter that has observed phishing attacks crossing one million in a quarter compared to the previous record of 888,585 attacks in 2024 Q2.

Even though lack of awareness is the main reason for growing phishing attacks, encouraging users to understand phishing attacks and browse accordingly is unrealistic. The online user needs continuous education as phishers try to identify vulnerabilities in existing techniques and design new phishing mechanisms such that phishing warnings are avoided. Hence, anti-phishing techniques were introduced to counter the phishing sites. The techniques include URLs, source code, images, and logos as the sources to classify the phishing sites. However, online users are still trapped in the attacker's plans and revealing sensitive information. Hence, in this work, we propose an efficient and lightweight model to detect phishing sites with significant performance and low latency.

Multiple anti-phishing techniques have been developed to tackle this issue, including blacklist-based systems, source code analysis, and visual similarity detection. While these approaches have been achieved, they have some limitations. Blacklist-based methods are ineffective against zero-day phishing sites, source code analysis struggles with dynamically generated content, and visual similarity approaches can be computationally expensive, particularly for resource-constrained mobile devices. These challenges underscore the need for lightweight, real-time, and highly accurate phishing detection mechanisms tailored to mobile environments.

This research addresses these challenges by introducing Phish-Jam, an ensemble-based approach specifically designed for detecting phishing sites on mobile devices. The proposed model leverages the power of ML to analyze URLs directly, bypassing the need for page content or heavy computation. The methodology incorporates handcrafted URL features, transformer-based text embeddings, and DL models to capture diverse patterns indicative of phishing activity. A super learner ensemble enhances accuracy and robustness, combining predictions from multiple ML algorithms.

The primary objective of Phish-Jam is to provide a scalable, efficient, and accurate solution for phishing detection on mobile devices. The proposed model is designed to operate effectively within the resource constraints of mobile platforms, ensuring quick and reliable detection without compromising device performance. Experimental evaluations demonstrate the effectiveness of the proposed approach, achieving high accuracy, precision, and recall, thus addressing the gaps in existing solutions.

Phish-Jam contributes to the growing field of mobile cyber security by focusing on mobile-specific challenges and leveraging advanced ML techniques. It offers a practical and reliable solution to safeguard users against phishing attacks, paving the way for more secure mobile internet usage in an increasingly digital world.

The following are the significant contributions of this work:

- This work proposes a robust phishing detection framework that analyzes and classifies URLs without relying on external metadata or webpage content. By focusing solely on URLs, the proposed model reduces vulnerability to zero-day attacks and drive-by downloads, addressing critical limitations of traditional anti-phishing techniques.
- The proposed work combines handcrafted URL features with powerful text embedding transformers and DL model features. This integration captures both structural patterns and semantic nuances in URLs, enhancing the detection capability of the proposed model.
- A novel super learner ensemble is proposed, which aggregates predictions from diverse ML algorithms. This ensemble approach leverages the strengths of individual classifiers, achieving superior performance in accuracy, precision, and F1 score.
- The proposed work incorporates DL models alongside traditional ML approaches, achieving state-of-the-art results. DL models complement the ensemble model, ensuring high reliability and robustness in phishing URL classification.

The proposed model has some advantages over existing works and is given as follows.

- *Independent of the third-party features* The proposed model is independent of external features from third-party sources, such as page ranking, domain age, and search engine indexing, as it does not rely on these features for detection.
- *Drive-by downloads independently* Because the model does not require visiting the actual website to extract features, it eliminates the risk of unintentionally downloading malicious software from web pages.
- *Client-Side adaptability* The URL features extracted in our proposed model are lightweight, making them easily adaptable for client-side implementation.
- *Target-independent detection* In contrast to image-based methods, our approach does not depend on a stored database of legitimate target sites, allowing it to detect phishing sites aimed at any legitimate website.
- The proposed technique utilizes semantic extraction of transformer embeddings to capture textual semantic features, aiding in identifying phishing websites that exhibit similar textual content.

This paper mainly includes six parts. The second chapter provides a review of relevant literature. The third chapter presents the proposed methodology. The fourth chapter discusses the experimentation and results. The fifth chapter deals with a discussion of our proposed model. The sixth chapter summarizes the conclusion and future work.

Related works

The challenge of detecting phishing URLs has prompted extensive research into various methodologies, from traditional heuristic-based approaches to advanced ML and DL techniques. This literature survey provides an

overview of key studies and developments in this field, highlighting the evolution of methods and the current state-of-the-art.

Traditional heuristic based approaches of phishing detection

Multiple studies have proposed ML methods for detecting phishing URLs. These models^{4,5} typically extract features from URLs, landing pages, and hosting details, which are then used to train classifiers to distinguish phishing URLs from legitimate ones. Shahoo et al.⁶ categorize these features into four types: blacklist features, lexical features, host-based features, and content-based features. Blacklist features identify URLs or slight variations listed on blacklists^{7,8}. URL lexical features, applied in studies like^{9–12}, examine words separated by special characters, along with host-based properties such as domain and IP attributes. Content-based features, including HTML and JavaScript elements^{13–17}, as well as textual and visual content^{18–21}, have also been widely utilized. Other studies incorporate hyperlink information on webpages^{22,23}, for example, by calculating the ratio of frequent anchor links to the total links on the page. Additionally, Tan et al.²⁴ explored graph-based methods, leveraging webpage hyperlinks and link structures. Other works use supplementary features from third-party sources like domain age data from WHOIS or page rank from the Alexa top domains list^{5,15,17,25,26}.

URL-based phishing detection

Apart from URL features, many of the previously mentioned features are difficult to obtain in real-time and can be easily bypassed by evasion methods²⁷. Moreover, extracting content features requires background access to the phishing webpage, which poses risks such as unintentional malware downloads and potential countermeasures, even in automated systems. Additionally, collecting hosting information is complicated for phishing sites with short lifespans. As a solution, URL-based feature approaches have been suggested as a complementary approach. These methods avoid reliance on third-party features, mitigate the risk of unintended malware downloads, and are not susceptible to web page evasion tactics like cloaking. They are also effective in detecting phishing sites hosted on compromised legitimate websites. This paper primarily investigates the phishing detection technique, which has been explored in previous research works^{28–32} and broadly categorized into two types.

Handcrafted features-based phishing detection

These approaches extract manually designed features from URLs, including the length of the URL, number of words, occurrence of special characters, and count of subdomains. ML algorithms such as Random Forest (RF) and Support Vector Machines (SVM) are then trained using these extracted features^{28,33,34}. Significant research in this area includes CatchPhish²⁸ and the work by Sahingoz et al.³⁴, which achieved accuracies of 98.25% and 97.98%, respectively. However, both studies evaluated their models exclusively on the datasets they gathered and introduced in their respective papers.

Embedding representations-based phishing detection

Recent advancements in DL have led to the development of various DL-based models for identifying phishing URLs. Yuan et al.³⁵ introduced a technique that converts URLs into character embeddings using a SkipGram language model³⁶, which are then used to train ML models such as XGBoost (XGB), Logistic Regression (LR), and RF. In a different approach, Rao et al.³⁷ employed URL word embeddings as input for an LSTM architecture, from which they extracted a feature layer to build an ensemble of SVMs.

Additional research has focused on creating complete DL pipelines. Both URLNet and the study by Aljofey et al.^{29,38} utilize URL embedding representations to train CNN classifiers. URLNet²⁹ begins by initializing randomly and training embeddings for each word in a URL, separated by special characters. It then generates embeddings for characters within each word and combines these with the word-level embeddings through element-wise addition. These combined embeddings and character-level embeddings are processed through a CNN model for final classification. Similarly, Aljofey et al.³⁸ trained a CNN model for phishing detection using only character-level embeddings. URLNet achieved a true positive rate of 95.58% and a false positive rate of 0.1%, while Aljofey et al. achieved an F1-score of 95.13%.

Maneriker et al.³⁰ introduced URLTran, a model that leverages state-of-the-art transformer architectures to identify phishing URLs. In contrast to previous techniques, URLTran fine-tunes a pre-trained BERT model³⁹ using sub-word tokenized URLs. This method enables the model to learn contextual relationships between URL tokens. The tokenized input is processed through the BERT model, and the resulting embeddings are utilized to train a feed-forward neural network that generates phishing predictions. URLTran demonstrated substantial improvement over URLNet, achieving a 21.9% relative increase in true positive rate while maintaining a low false positive rate of 0.01%.

ML and DL based approaches

Various ML algorithms, including Support Vector Machine (SVM), Random Forest (RF), Logistic Regression (LR), and Bayesian network (BN), are employed in these approaches^{5,34,40,41} to analyze hidden patterns within the features for classifying phishing websites. These features are extracted using heuristic methods, utilizing either the URL, source code, or third-party sources.

Several studies^{29,42–44} have employed various DL algorithms for URL classification. These methods include Deep Neural Network (DNN), Recurrent Neural Network (RNN), Long Short Term Memory (LSTM), Deep Belief Network (DBN), and Convolution Neural Network (CNN).

Mobile based approaches

The proposed model approach aligns with mobile-based antiphishing techniques, so this section highlights some of the latest and most popular methods in this category. Han et al.⁴⁵ introduced a method for detecting

phishing websites using pre-stored Login User Interface (LUI) information on mobile devices. Their approach involves a browser plug-in that compares the LUI information of suspicious websites with the pre-stored LUI data to identify phishing sites.

MobiFish, an automated antiphishing system for mobile devices, was proposed in Ref.⁴⁶. This system identifies phishing websites and malicious applications in mobile devices by verifying the actual identity against the actual identity using Optical Character Recognition (OCR). OCR is employed to extract text from the website, which is used to determine the claimed identity, while the actual identity is derived from the URL. If a mismatch between the two identities is detected, the system warns the user.

A URL-based approach for detecting phishing websites on mobile devices was proposed in Ref.⁴⁷. This method analyzes the frequency of phishing-related features to determine the status of a website. These URL-based features are input into an SVM model to identify phishing attacks.

A defense mechanism was introduced in Ref.⁴⁸ that monitors keystrokes and alerts users when sensitive information is entered into a malicious mobile application. This approach relies on a whitelist of trusted applications and their associated userIDs to verify the legitimacy of the application.

Amrutkar et al.⁴⁹ developed a Firefox extension called kAYO to detect phishing websites on mobile devices. The authors extracted different static features from the content of HTML, JavaScript, URLs, and mobile-specific features. LR is then applied to classify websites as either phishing or legitimate.

Chorghe et al.⁵⁰ proposed a method for detecting phishing websites on Android mobile devices. The approach involves extracting the URL from the browser, then a static analysis of the URL, HTML extraction, and the retrieval of lexical and third-party-based features. These features are input to an SVM to classify the URLs.

Ndibwile et al.⁵¹ introduced a method for detecting mobile phishing pages using deceptive login simulation. This approach was implemented as an Android application called UnPhishMe, which submits fake credentials to the login form of a suspicious website. The website's legitimacy is determined by verifying the authentication process with the fake credentials. Authentication is calculated by comparing the hash codes of the suspicious URL before and after the login process.

Rao et al.³⁷ developed a mobile application named PhishDump to classify websites on mobile devices as either legitimate or phishing. PhishDump utilizes a multi-model approach, combining LSTM models and an SVM classifier. By focusing on attribute extraction from URLs, PhishDump offers several advantages over other methods, including faster computation and language independence.

BERT and ELECTRA are prominent transformer models in the field of Natural Language Processing (NLP). Haynes et al.⁵² utilized these models to learn feature representations directly from URL text. Both BERT and ELECTRA show comparable performance, delivering promising results in detecting malicious URLs on mobile devices.

Jain et al.⁵³ introduced APuML, an efficient mobile-based system for detecting malicious websites. The system operates through multiple stages, including data feed, DNS analysis, and ML. The authors show that APuML achieves high detection accuracy while maintaining a low response time.

Dhanavanthi et al.⁵⁴ presents an approach to web phishing detection using DL techniques, specifically LSTM and Gated Recurrent Unit (GRU) networks. This paper emphasizes the increasing sophistication of phishing attacks and the need for effective detection methods that can operate efficiently on resource-constrained devices, such as mobile phones and Raspberry Pi.

Gupta et al.⁵⁵ proposed a DL based solution for detecting mobile phishing attacks using the GoogLeNet model. The model analyzes webpage screenshots and alerts users if a phishing attempt is detected. The GoogLeNet model was chosen for its effective multi-class image recognition capabilities. The proposed model achieved an impressive accuracy of 97.04%, outperforming traditional ML models like LR, DR, and SVM.

Ponsam et al.⁵⁶ proposed a phishing detection system called PhishShield that leverages ML models within the Flask framework. The system employs Bernoulli Naive Bayes and Multinomial Naive Bayes algorithms to classify URLs as Phishing or Legitimate. The phishShield extracts key features such as IP address details, URL characteristics, and domain-related attributes. PhishShield's user interface, built using Flask, CSS, and Bootstrap, ensures a user-friendly experience for quick URL assessments, offering a valuable solution for combating phishing attacks.

Previous research on phishing URL detection has utilized either handcrafted features or DL features, such as word-level embeddings and transformer-based representations. Table 1 provides a comparative summary of these mobile-based phishing detection techniques, highlighting their methodologies and key contributions. In this work, we propose a novel hybrid model that integrates feature vector representations of both handcrafted and DL features. These combined feature vectors are then processed by an ML model for classification. Additionally, we propose a super learner ensemble, which aggregates predictions from various ML algorithms to enhance classification performance. Our proposed model aims to deliver more effective and adaptive phishing URL detection, which is particularly suited for mobile device deployment.

Proposed work

Attackers craft phishing URLs to appear legitimate, using obfuscation techniques to deceive users into revealing personal information. This paper aims to detect phishing sites in real-time using lightweight URL-based features without visiting the website. Before presenting the proposed model, we discuss URL components in the following. The Fig. 1 illustrates the structure of a URL broken down into its components:

1. *Protocol* `http://` indicates the communication protocol used to access the resource.
2. *Subdomain* `www` is the subdomain often used for web services within a larger domain.
3. *Root Domain* `deletion.us` represents the primary root domain name and top-level domain (`.us`).
4. *Directory* `/home` indicates a folder or directory path on the server.

	Features	Key description	Model
Wu et al. ⁴⁶	OCR-based identity verification	Extracts text from website and verifies claimed vs actual identity	OCR based model
Orunsolu et al. ⁴⁷	URL-based features	Determine the status of a website based on frequency analysis of extracted phishing features	SVM
Amrutkar et al. ⁴⁹	Extracts static features from HTML, JavaScript, and URLs	Identify a range of new static features that highly correlate with mobile malicious webpages	LR
Chorghe et al. ⁵⁰	URL static analysis with HTML and third-party features	The approach involves extracting the URL from the browser	SVM
Ndibwile et al. ⁵¹	Deceptive login simulation using fake credentials	Compares hash codes before and after login	Hash code
Rao et al. ³⁷	Multi-model approach for phishing detection	Combines LSTM models and an SVM classifier for faster computation	LSTM + SVM
Haynes et al. ⁵²	Transformer models for URL feature	Utilizes BERT and ELECTRA models for text-based features	BERT, ELECTRA
Jain et al. ⁵³	Extracts all the static and site popularity features	Combines data feed, DNS analysis, and ML techniques	RF
Dhanavanthi et al. ⁵⁴	DL techniques for resource-constrained devices	Uses LSTM and GRU networks for efficient detection on mobile phones and Raspberry Pi	LSTM, GRU
Gupta et al. ⁵⁵	Web page based features	DL based solution for detecting mobile phishing attacks using the GoogLeNet model	LR, DR, SVM
Ponsam et al. ⁵⁶	IP address features, URL features, and domain features	PhishShield user interface, built using Flask, CSS, and Bootstrap	Bernoulli NB and Multinomial NB models

Table 1. Summary of mobile-based phishing detection methods.

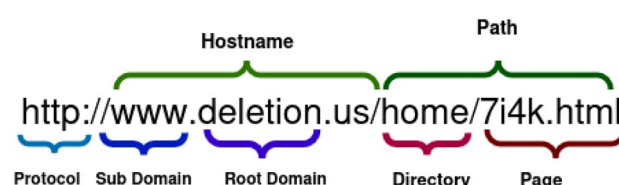


Fig. 1. Basic URL structure diagram.

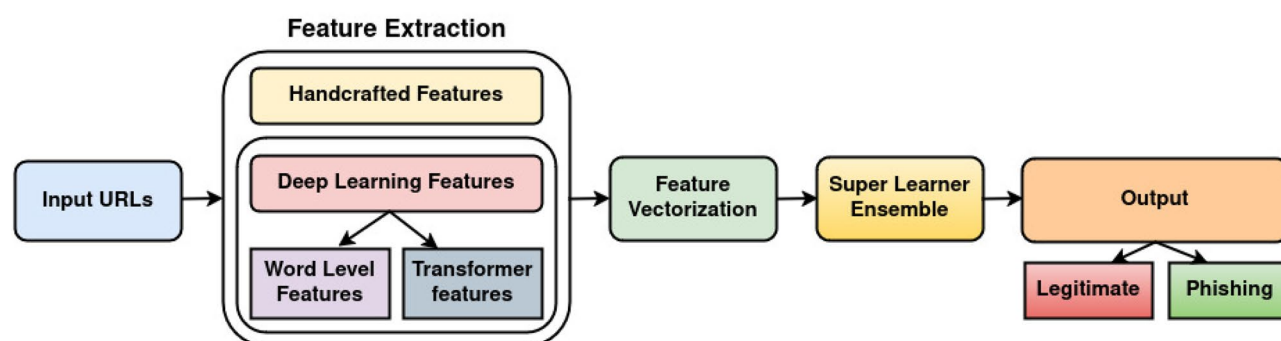


Fig. 2. Architecture of proposed work.

5. *Path* /home/7i4k.html indicates the specific resource path on the server.
6. *Page* 7i4k.html specifies the actual HTML page or resource being accessed.

The architecture of the proposed work is shown in Fig. 2. The model takes the URL as input and classifies it as legitimate or phishing as output. The suspicious URL is fed to the feature extraction module, which extracts features using DL architectures such as LSTM and Transformers. Also, handcrafted features are extracted from the URL to complement the detection. Various ML algorithms are used to identify the best algorithm for the classification of URLs. To include the diverseness of each classifier, a super learner ensemble is proposed for the classification task.

- *Input URLs* The raw input data consists of both phishing and legitimate URLs, which need to be classified as either legitimate or phishing.
- *Feature extraction* This component extracts features from the given input URL fed to the ML model for classification. The features are extracted from various parts of the URL. Firstly, handcrafted features such as count-based and Boolean-based from the hostname, primary domain, and sub-domain of the URL are extracted.

Secondly, DL features are used, such as feeding tokenized words to DL architectures and transformers for feature extraction.

- *Handcrafted features* Handcrafted features include a count of digits, dots, hyphens in the subdomain and primary domain, the ratio of digits and characters in the subdomain and primary domain, the presence of HTTPS, special characters (@), the total length of the primary domain, and subdomain, etc. A detailed list of the handcrafted features used in this work is presented in Table 2.
- *Deep learning features* The URL is tokenized for DL-based features, and word embedding techniques are applied before applying DL algorithms such as LSTM. As mentioned, before the extraction of handcrafted or DL-based features, word tokenization is done as a preprocessing step. We use several delimiters, such as dot, slash, hyphen, underscore, etc., to split the URL into tokens, which sometimes might result in loss of information. Because the attacker might use different characters to imitate the legitimate URLs. Hence, we also use a segmenter with a Twitter corpus to identify meaningful tokens in the URLs, and the words are fed to either handcrafted or DL-based feature extraction. One of the other feature extraction modules includes a transformer such as Bidirectional Encoder Representations from Transformers (BERT), where the embedding from the transformer is used to generate features.

LSTM LSTM is a specific type of Recurrent Neural Network (RNN) architecture designed to model temporal sequences and their long-range dependencies more precisely than traditional RNNs. LSTMs were first introduced by Hochreiter and Schmidhuber in 1997, and their ability to effectively handle long-term dependencies makes them extremely useful in several applications, including sequence data patterns and time series analysis.

Traditional RNNs face a significant challenge in learning long-term dependencies due to the vanishing or exploding gradient problem. As the gradient of the loss function decays exponentially with the length of the input sequence, RNNs struggle to connect information from earlier time steps to later ones. In other words, if an RNN is processing a sentence, it may “forget” the first part of the sentence before it finishes processing the entire input.

LSTM networks, on the other hand, effectively mitigate this problem. By incorporating a “memory cell” that can maintain information in memory for long periods of time, an LSTM unit can remember or forget specific values based on the outputs of its gates. The design of these gates allows LSTMs to propagate gradients back through time more effectively and thus learn from long-term dependencies.

The LSTM unit comprises a memory cell, which consists of an input gate, an output gate, and a forget gate. The memory cell is responsible for “remembering” values over arbitrary time intervals, hence the word “memory” in LSTM. Each of the three gates can be considered a “conventional” artificial neuron,

Feature name	Description	Relevance to phishing detection
domain_length	Length of the domain part of the URL	Phishing domains are often longer to mimic or confuse legitimate sites
num_subdomains	Number of subdomains in the URL	Phishers use multiple subdomains to obscure the true domain
hyphen_in_domain	Presence of hyphens in the domain name	Hyphens are often used in deceptive domain names
numeric_in_domain	Presence of numeric characters in the domain	Numbers may be used to imitate legitimate services
phishing_tld	Use of TLDs known for phishing activity	Certain TLDs are more commonly abused by phishers
is_ip_address	Domain is an IP address instead of a name	Raw IPs are used to avoid domain blacklisting
number_of_digits	Total number of digits in the URL	High digit counts may indicate obfuscation
uncommon_tld_in_subdomains	Uncommon TLD-like patterns in subdomains	Used to mimic legitimate domains and mislead users
longest_subdomain_length	Length of the longest subdomain	Longer subdomains may conceal malicious components
url_with_port	Presence of a port number in the URL	Non-standard ports are common in phishing attempts
path_length	Length of the path in the URL	Abnormally long paths may be suspicious
num_directories	Number of directories in the URL path	Deep directory structures mimic legitimate sites
special_chars_in_path	Number of special characters in the path	Special characters may indicate manipulation or encoding
num_query_parameters	Number of query parameters in the URL	Excessive parameters can hide phishing intentions
special_chars_ratio_url	Ratio of special characters in the entire URL	High ratios may suggest complexity or obfuscation
special_chars_ratio_domain	Ratio of special characters in the domain	Detects suspicious or malformed domains
special_chars_ratio_path	Ratio of special characters in the path	Reveals irregularities and encoded content
digits_ratio_path	Ratio of digits in the path	Excessive numbers in paths can be deceptive
digits_ratio_url	Ratio of digits in the entire URL	Overall numerical density as a risk indicator
digits_ratio_domain	Ratio of digits in the domain	Numeric spoofing to resemble legitimate services
num_encoded_char	Number of encoded characters (e.g., %20)	Encoding used to evade detection
uses_https	Whether the URL uses HTTPS	Legitimate sites usually use HTTPS; phishing often omits it
entropy_url	Shannon entropy of the entire URL	High entropy suggests randomness or obfuscation
entropy_path	Shannon entropy of the path	Detects abnormal randomness in paths
entropy_domain	Shannon entropy of the domain	Helps identify suspicious and complex domain patterns

Table 2. List of handcrafted features used in the proposed model.

as in a multilayer perception, they compute the activation (using an activation function) of a weighted sum. Intuitively, they can be understood as regulators of the flow of values that goes through the connections of the LSTM, hence the denotation “gate”.

The input gate determines how much incoming information should be stored in the memory cell. The input gate uses a sigmoid activation function that outputs a value between 0 (ignore the information) and 1 (keep all the information). Next, a tanh layer creates a vector of new candidate values, $\tilde{C}[t]$, that could be added to the state. The input gate employs the following equation, denoted as Eq. 1.

$$\begin{aligned} i[t] &= \sigma(Wi.[h[t-1], x[t]] + bi) \\ \tilde{C}[t] &= \tanh(Wc.[h[t-1], x[t]] + bc). \end{aligned} \quad (1)$$

The forget gate decides which information to discard from the cell state. It looks at $x[t]$ and $h[t-1]$ at time t and outputs a number between 0 and 1 for each number in the cell state $c[t-1]$. The value one means “keep everything”, while zero means “forget everything”. The forget gate is represented by Eq. 2

$$f[t] = \sigma(Wf.[h[t-1], x[t]] + bf). \quad (2)$$

In an LSTM, the previous cell state $c[t-1]$, is updated to form the new cell state $c[t]$. Applying the forget gate $f[t]$ to the old cell state effectively decides what information to retain and what to discard. We assume the new candidate values $\tilde{C}[t]$, which have been scaled by the input gate $i[t]$ to determine how much of this new information should be used to update the state. The output will be a modified version of our cell state. Initially, a sigmoid activation function is applied to decide which parts of the cell state we intend to output. Then, we take the cell state through a tanh function, normalizing the values to a range between -1 and 1 , and multiply it by the output of the sigmoid function, thereby ensuring that only the desired elements of the cell state are output. The updated state and output gate are as shown in Eq. (3)

$$\begin{aligned} c[t] &= f[t] * c[t-1] + i[t] * \tilde{C}[t] \\ o[t] &= \sigma(Wo.[h[t-1], x[t]] + bo) \\ h[t] &= o[t] * \tanh(c[t]). \end{aligned} \quad (3)$$

Multi-head self attention (MHSA) The inspiration for the attention mechanism is drawn from human visual perception. Due to the bottleneck of information processing capacity limitations, humans tend to concentrate on specific aspects of information while disregarding others selectively. This selective focusing process is commonly known as the attention mechanism. Over recent years, the attention mechanism has become a cornerstone in various tasks associated with Natural Language Processing (NLP) that rely on DL techniques.

Input information can be represented in the form of key-value pairs, where the “key” assists in calculating the attention distribution, and the “value” contributes to generating the selected information. There exist several variations of the attention mechanism. Multi-head attention leverages multiple queries to compute the selection of various information from the input parallelly. Each attention weight emphasizes a distinct part of the input. Self-attention assigns equal value to the key, focusing primarily on the internal structure of a query. Google recently fused these two attention mechanisms, resulting in what is now known as MHSA⁵⁷. MHSA is an attention mechanism that links different positions of a single sequence to compute its representation. A depiction of multi-head attention is provided in Fig. 3. The architecture consists of linear projection, attention layer, concatenation and final linear projection.

Linear projection of each head, we start with queries (Q), keys (K), and values (V) that are derived from the input. These are transformed using learned linear projections specific to each head. This can be represented in Eq. (4)

$$head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (4)$$

where Q, K, V are input query, key, and value matrices, W_i^Q, W_i^K, W_i^V are the weight matrices of the i -th head for query, key and value, respectively.

The transformed Q, K , and V are then fed into the scaled dot-product attention mechanism. The output for each head i is calculated as Eq. (5)

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (5)$$

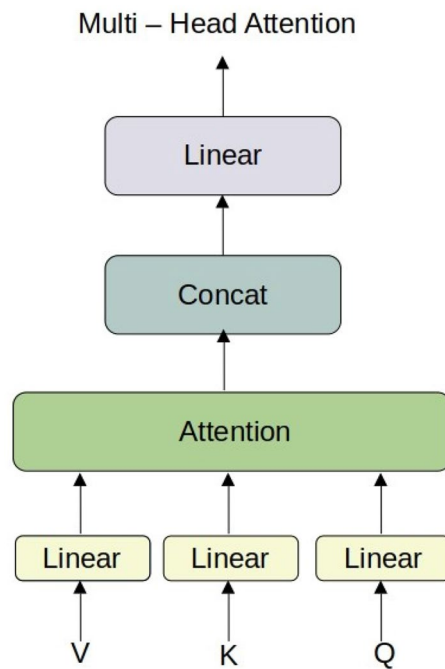


Fig. 3. Multi-head attention architecture.

where d_k is the dimension of the key vectors (this scaling helps to stabilize the learning), and the softmax function ensures the output weights sum to 1. Concatenation of the individual attention head outputs is then concatenated. The softmax function is calculated as Eq. 6

$$\text{softmax}(X)_{ij} = \frac{\exp(X_{ij})}{\sum_j \exp(X_{ij})}. \quad (6)$$

The final linear projection of the concatenated output is then passed through another learned linear projection to result in the final values. The final output is calculated as Eq. 7

$$\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (7)$$

where W^O is another learned weight matrix.

The whole process allows the model to capture different types of information from the input data. Using multiple attention heads, the model can learn to focus on different parts of the input sequence in each head, leading to richer representations.

Transformer-based word embedding technique The advanced transformer-based models have shown state-of-the-art performance in NLP. Developed by Vaswani et al.⁵⁷ in the paper “Attention is All You Need,” transformers rely on self-attention mechanisms to process input sequences in parallel, making them efficient and effective for NLP. These models generate context-aware embeddings of the URLs by processing the entire input sequence of characters in the URL and providing deep contextual understanding. The contextual word embedding representation of a word depends on the entire sentence. The following is the transformer-based word embedding technique.

BERT algorithm This model generates embeddings by considering both the left and right context in all layers, capturing nuanced meanings and relationships. After tokenization, each token is mapped to a low-dimensional vector using the BERT model, which captures the word’s semantic meaning within the entire URL. Unlike traditional word embeddings, BERT considers the entire sequence of tokens, enabling it to capture context and understand word relationships. It helps analyze URLs and identify subtle clues that indicate a phishing attack. The model can continuously adapt to new phishing tactics by fine-tuning BERT with phishing data.

- **Feature vectorization** We integrated features from three different sources: 25 handcrafted statistical features, 768-dimensional contextual embeddings from the transformer (BERT), and 128-dimensional features extracted via the proposed multi-attention mechanism. These features were concatenated to form a comprehensive 921-dimensional feature vector. This unified feature representation was then fed to various ML algorithms to classify phishing URLs. Multiple ML algorithms are experimented with to identify the best algorithm for the chosen selected dataset. The ML algorithms are chosen to capture diverse information for classification. The ML algorithms include XGBoost (XGB), SVM, KNeighbour (KNN), LR and RF. Also, An ensemble of

these algorithms, the super learner ensemble, is proposed to enhance classification performance further. By leveraging the strengths of individual classifiers, this ensemble model aims to achieve superior accuracy and robustness in phishing URL detection.

- **Super learner ensemble** The working of the super learner is given in Fig. 4. It involves a k-fold data split where multiple models are applied to the split data and combined by training a meta-learner to produce predictions based on the weak learners. The meta-learner takes predictions as features and ground truth values for training the model. It finds the path to combine the predictions best to achieve a better output prediction.

The steps involved in implementing the super learner ensemble:

1. Split the dataset into 80:20 for training and testing instances.
2. Use K-fold cross-validation with the training data.
3. For each fold, apply baseline models and output the predicted probabilities.
4. Store the predictions from each fold and each model to generate the new data.
5. Stack these predicted probabilities and the ground truths for training with meta learner.
6. Test data on the baseline-trained models are used, and the predicted probabilities of baseline values are fed to the trained meta-learner for the final predictions.

Results

In this section, we conducted various experiments to detect phishing attacks. Firstly, we experimented with handcrafted features extracted from both phishing and legitimate sites. Secondly, We have experimented with the model of LSTM-based features and LSTM with multi-attention features for the classification. Thirdly, transformer-based features are extracted from the dataset for classification. Finally, combining all the features, including LSTM with multi-attention, handcrafted, and transformer-based features, is used as the final feature vector, which is fed to the super learner ensemble for the classification. To ensure reproducibility, we provide list of hyperparameters used in our deep learning models in Tables 3 and 4. For traditional machine learning algorithms, we used the default hyperparameter settings provided by their respective implementations in Scikit-learn, as they yielded competitive baseline results. Also, we have made our code publicly available at <https://tinyurl.com/phishjamcode>

Dataset

The PhishDump dataset is used for all experiments where it contains 190,062 legitimate and 141,490 phishing URLs. The phishing URLs are collected from PhishTank and OpenPhish. Legitimate sites are obtained from the top 100 Google search results for each brand name given as a search query. These brand names are collected from PhishTank (206) and the Alexa database (top 2000 out of 1 million brand names). The details of the dataset are shown in Table 5.

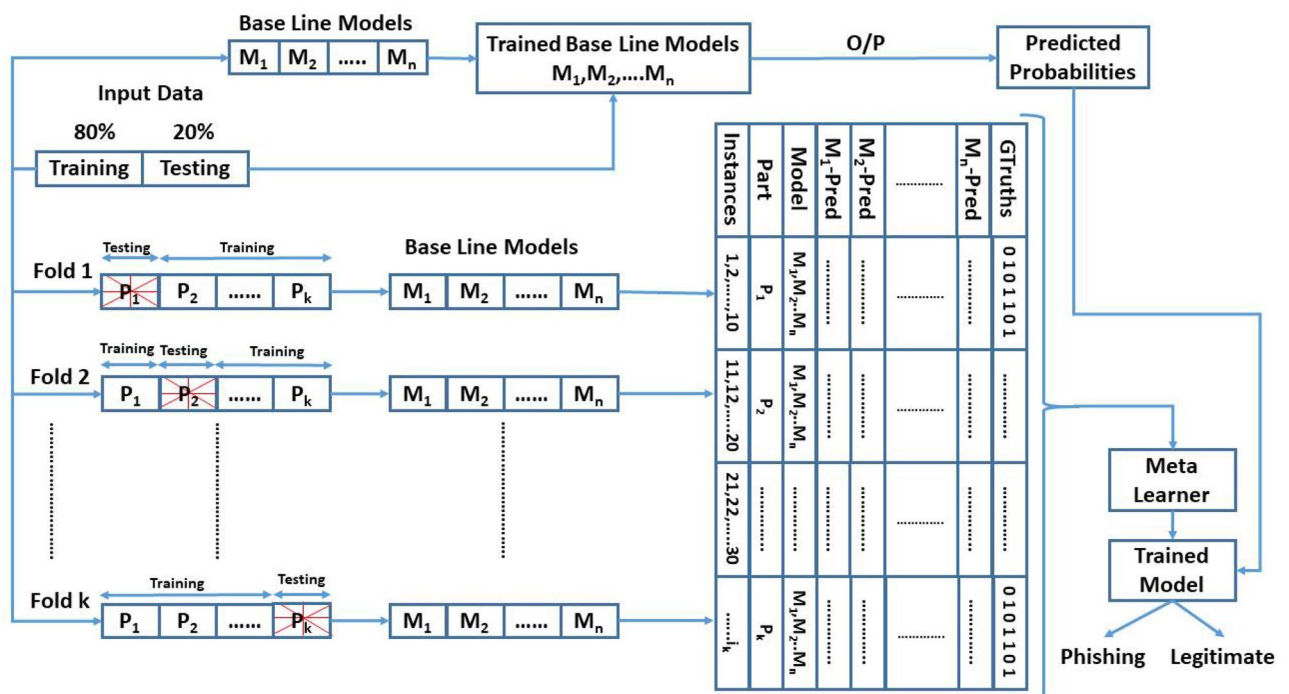


Fig. 4. Working of super learner ensemble.

Hyperparameter	Value
Maximum sequence length	75
Vocabulary size	$ word_index + 1$
Embedding dimension	100
Number of CNN filters	64
CNN filter sizes	[3, 4, 5]
LSTM units	128
Number of attention heads	4
Attention key dimension	32
Dropout rate	0.2
Loss function	Binary cross-entropy
Optimizer	Adam
Train-test split	80–20%

Table 3. Hyperparameter settings for the LSTM multi-attention model.

Hyperparameter	Value
Pretrained model	BERT-base uncased
Maximum sequence length	128
Batch size	64
Embedding dimension	768
Pooling method	Mean pooling over tokens
Train-test split	80–20%

Table 4. Hyperparameter settings for the BERT-based model.

Type	Source	Sites	URL
Legitimate	PhishTank	19510	http://phishtank.com/target_search.php
	Alexa database	170552	http://www.alexa.com/topsites
Phishing	OpenPhish	7212	http://openphish.com/
	PhishTank	134278	http://www.phishtank.com/developer

Table 5. The PhishDump dataset used for experimentation.

Evaluation metrics

We considered traditional metrics such as Recall, Precision, True Negative Rate (TNR), F1-Score, and Accuracy to evaluate the model's performance. Phishing sites are considered positive instances, and legitimate sites are negative instances. The model predicting phishing sites as phishing sites ($P \rightarrow P$) is considered as True Positives, whereas legitimate sites as legitimate ($N \rightarrow N$) are considered as True Negatives. Here, True Positive (TP) indicates the phishing URLs correctly identified as phishing, while True Negative (TN) represents legitimate URLs correctly identified as legitimate. False Positive (FP) refers to legitimate URLs incorrectly classified as phishing (false alarms), and False Negative (FN) refers phishing URLs incorrectly classified as legitimate (missed attacks). The total instances encompass all cases within the dataset. We use the following evaluation metrics to assess the robustness of our model:

- True Negative Rate (TNR) helps to measure how well the model avoids false alarms by correctly identifying legitimate URLs.

$$TNR = \frac{TN}{FP + TN} \times 100. \quad (8)$$

- Recall is critical for phishing detection, as it reflects the model's ability to detect actual phishing attempts (minimizing FN).

$$Recall = \frac{TP}{TP + FN} \times 100. \quad (9)$$

Metrics	ML1	ML2	ML3	ML4	ML5
Recall (%)	92.54	78.15	88.64	86.08	91.42
TNR (%)	90.76	81.10	86.40	80.07	90.73
Precision (%)	93.21	88.47	90.14	84.95	93.32
F1-Score (%)	92.87	82.99	89.39	85.51	92.36
Accuracy (%)	91.79	79.18	87.71	83.48	91.13
MCC (%)	83.19	57.16	74.81	66.30	81.83

Table 6. The results of the proposed model with different ML algorithms.

Metrics	LSTM	LSTM with multi-attention
Recall (%)	96.62	96.05
TNR (%)	96.39	97.19
Precision (%)	95.20	96.18
F1-Score (%)	95.91	96.12
Accuracy (%)	96.49	96.71

Table 7. The results of the proposed model with LSTM and LSTM with multi-attention features.

- Precision shows the proportion of predicted phishing URLs that are actually phishing, which is important to avoid unnecessary warnings.

$$Precision = \frac{TP}{TP + FP} \times 100. \quad (10)$$

- F1-Score balances precision and recall, ensuring that the model is both sensitive to phishing threats and precise in its alerts.

$$F1 - Score = 2 \times \frac{Pre \times Recall}{Pre + Recall} \times 100. \quad (11)$$

- Accuracy provides an overall measure but is interpreted cautiously due to class imbalance in phishing datasets.

$$Accuracy = \frac{TP + TN}{Total\ instances} \times 100. \quad (12)$$

- Matthews Correlation Coefficient (MCC) gives a more balanced evaluation, particularly when datasets are skewed, by considering all four confusion matrix categories.

$$MCC = \frac{(TP \cdot TN) - (FP \cdot FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (13)$$

Experiment 1: evaluation of Handcrafted features with different ML algorithms

In this section, the handcrafted features are fed to various ML algorithms such as XGB (ML1), SVM (ML2), KNN (ML3), LR (ML4), and RF (ML5) for the classification of phishing sites. This experiment aims to identify the best model and richness of the features. The results are shown in Table 6. The results show that XGB outperformed other classifiers with an accuracy of 91.79%, precision of 93.21%, and recall of 92.54%.

Experiment 2: evaluation of LSTM and LSTM with a multi-attention mechanism features for the classification

This section shows the effectiveness of LSTM-based features and the inclusion of a multi-attention mechanism for the classification. The results are shown in Table 7. The results show that LSTM with a multi-attention mechanism performed better than LSTM alone, with an accuracy of 96.71%, precision of 96.18%, and recall of 96.05%. The architecture of the designed LSTM includes a dense layer preceding the output layer. Also, we have used LSTM with multi-attention features to feed into different ML algorithms, as shown in Table 8. The results show that SVM outperformed other ML algorithms with an accuracy of 98.57%, precision of 98.99%, and F1-Score of 98.76%.

Metrics	ML1	ML2	ML3	ML4	ML5
Recall (%)	98.29	98.52	98.18	98.43	98.37
TNR (%)	97.99	98.64	98.37	98.13	98.37
Precision (%)	98.51	98.99	98.80	98.62	98.79
F1-Score	98.40	98.76	98.49	98.53	98.58
Accuracy (%)	98.16	98.57	98.26	98.31	98.37
MCC (%)	96.25	97.08	96.45	96.54	96.67

Table 8. The results of the proposed model with different ML algorithms using LSTM with multi-attention features.

Metrics	ML1	ML2	ML3	ML4	ML5
Recall (%)	94.51	95.32	96.96	94.56	93.16
TNR (%)	92.82	93.88	93.35	92.86	91.23
Precision (%)	94.69	95.47	94.93	94.72	93.53
F1-Score	94.60	95.39	95.93	94.64	93.34
Accuracy (%)	93.80	94.71	95.38	93.84	92.34
MCC (%)	87.31	89.18	90.61	87.40	84.33

Table 9. The results of the transformer-based embedding features with different ML classifiers.

Metrics	ML1	ML2	ML3	ML4	ML5	SL
Recall (%)	98.42	95.40	95.26	98.42	98.33	98.98
TNR (%)	98.24	94.58	93.20	97.98	98.34	98.85
Precision (%)	98.70	96.02	94.93	98.50	98.78	99.15
F1-Score	98.56	95.71	95.10	98.46	98.55	99.07
Accuracy (%)	98.35	95.05	94.38	98.23	98.34	98.93
MCC (%)	96.62	89.88	88.52	96.39	96.60	97.81

Table 10. The results of the combined features with different ML classifiers and super learner ensemble.

Experiment 3: evaluation of transformer based features

In this section, we have experimented with the model embedding URLs with transformers, namely BERT. The transformer-based embedding features are fed to multiple ML algorithms, and the results are shown in Table 9. The Table shows that the model with KNN achieved an accuracy of 95.38%, precision of 94.93%, and recall of 96.96%. The transformed-based model outperforms the model with handcrafted features with a significant difference in performance metrics.

Experiment 4: evaluation of proposed model with combined features

This section evaluates the combination of handcrafted, LSTM with multi-attention embeddings, and transformer-based embedding features for the classification. The combined features are generated as a feature vector and further fed to ML algorithms and super learner ensemble. The results are shown in Table 10. The results show that the XGBoost model with combined features achieved better performance compared to other ML models except for the super learner ensemble, achieving an accuracy of 98.34%, precision of 98.78%, and F1 Score of 98.55%. Also, the super learner ensemble achieved the highest accuracy of 98.93%, precision of 99.15%, MCC of 97.81% and F1-score of 99.07%.

Experiment 5: comparison of proposed work with existing works

We compared our proposed work with existing works that utilize DL and ML algorithms for phishing URL detection. We implemented the methods proposed by Bahnsen et al.⁴³, Patgiri et al.⁵⁸, Sahingoz et al.³⁴, and Rao et al.³⁷, applying the same dataset used for our proposed work. Our proposed super learner ensemble model was also evaluated on the same dataset to compare its performance with those of these existing works. These works were selected for comparison due to their focus on DL and ML algorithms for URL classification, which aligns similarly with our approach. Unlike content-based phishing detection methods, these techniques detect phishing sites based on URLs. As shown in the experimental results in Table 11, it is observed that our proposed super learner ensemble outperformed the existing works across all evaluation metrics. It achieved an accuracy of 98.93%, TNR of 98.85%, recall of 98.98%, F1-Score of 99.07%, and a precision of 99.15%, which shows the significance of detecting phishing sites over existing works.

Reference	Recall (%)	TNR (%)	Precision (%)	F1-Score	Accuracy (%)
Bahnsen et al. ⁴³	96.18	95.67	96.81	96.49	95.96
Sahingoz et al. ³⁴	99.00	96.92	97.00	98.00	97.98
Patgiri et al. ⁵⁸	84.76	88.94	84.99	84.87	87.16
Rao et al. ³⁷	97.31	97.28	97.99	97.65	97.30
Proposed work	98.98	98.85	99.15	99.07	98.93

Table 11. Comparison of proposed work with existing works.

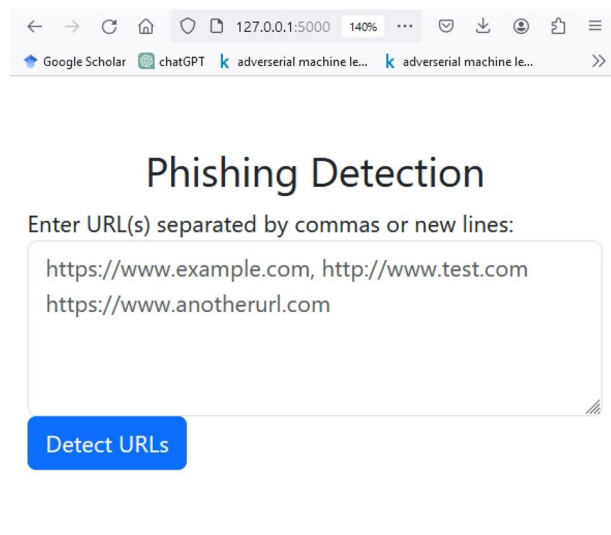


Fig. 5. Homepage of proposed model.

Discussion

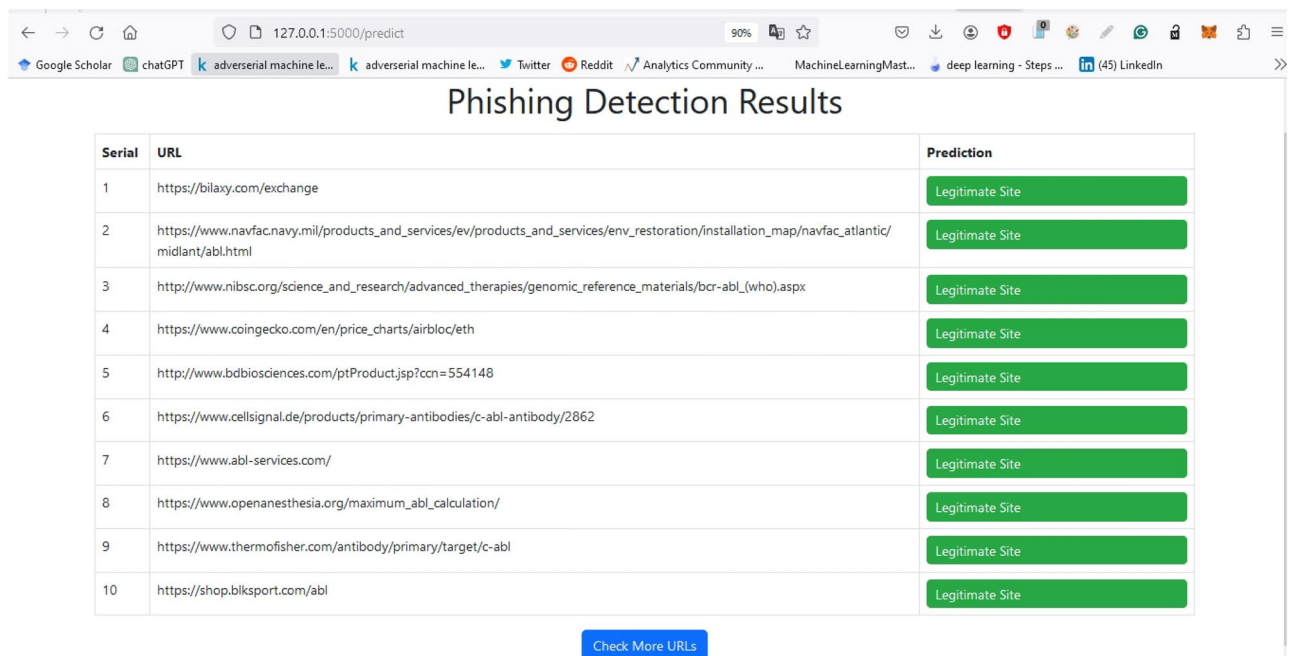
The primary aim of the proposed super learner ensemble is to provide real time protection against phishing sites on mobile devices. A lightweight and efficient mechanism is required for the early detection of phishing sites. Hence, we developed Phish-Jam, an Android application implemented on a Google Nexus 5 running the Android 14 operating system. We have developed a REST API that accepts the URL as input and returns its status (phishing or legitimate) as output. This API, built using Flask, a Python microframework, runs on an Ubuntu system with an Intel Xeon 16-core, 2.67 GHz processor and 16 GB of RAM. The application utilizes the GET method to transmit the URL status to its interface.

The User Interface (UI) of Phish-Jam, shown in Fig. 5, includes an input field for URL entry and a phishing detection page displayed as the default web view. Once the URL status is determined, the result (phishing or legitimate) is presented to the user, as shown in Figs. 6 and 7. We measured the response time to assess the application's real-time adaptability and found that Phish-Jam completes the detection process in an average time of 480 ms. This low response time is achieved by parallelizing the execution of eight models across multiple cores in the system. The output labels from the respective eight models are ensembled to return the final output as either legitimate or phishing.

While Phish-Jam focuses on URL analysis rather than source code, its performance depends on a few factors, such as Internet speed and mobile device RAM. Our experiments used a network bandwidth of 6 Mbps and 4 GB of RAM. A comparison of response times with existing techniques with Phish-Jam, as shown in Table 12, reveals that Phish-Jam offers a significantly faster detection rate, making it more suitable for real-time use on mobile devices.

The proposed super learner ensemble integrates DL, transformer-based, and ML classifiers to enhance performance, and it was observed that ML and DL algorithms have both pros and cons. While DL models excel in feature generation without prior knowledge, they are often challenging to interpret the actual working of the DL-based models. In contrast, ML classifiers are more transparent in their operation but require prior knowledge to design relevant features. The combination of these approaches in the super learner ensemble addresses these limitations, resulting in improved accuracy. Experimental results confirm that the super learner ensemble outperforms existing techniques that rely on ML or DL models individually for URL classification.

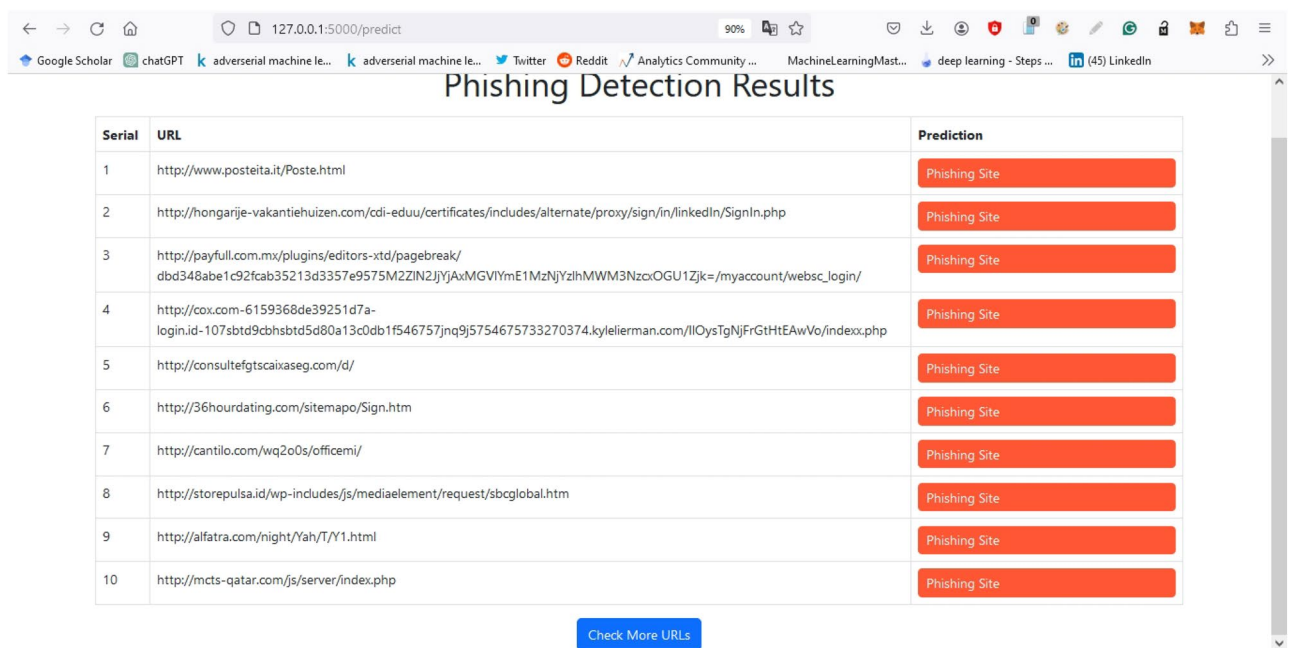
Although Phish-Jam demonstrates strong performance in phishing detection, it can potentially be circumvented if attackers modify the URL structure to closely mimic legitimate domains. Additionally, the system may face challenges in detecting phishing attempts that use shortened URLs (e.g., Bitly, Goo.gl, TinyURL), where the actual domain remains hidden. Furthermore, the model has not been tested on non-English URLs and is currently designed to work primarily with URLs containing English characters. In future work, we aim



Serial	URL	Prediction
1	https://bilaxy.com/exchange	Legitimate Site
2	https://www.navfac.navy.mil/products_and_services/ev/products_and_services/env_restoration/installation_map/navfac_atlantic/midlant/abl.html	Legitimate Site
3	http://www.nibsc.org/science_and_research/advanced_therapies/genomic_reference_materials/bcr-abl_(who).aspx	Legitimate Site
4	https://www.coingecko.com/en/price_charts/airbloc/eth	Legitimate Site
5	http://www.bdbiosciences.com/ptProduct.jsp?ccn=554148	Legitimate Site
6	https://www.cellsignal.de/products/primary-antibodies/c-abl-antibody/2862	Legitimate Site
7	https://www.abl-services.com/	Legitimate Site
8	https://www.openanesthesia.org/maximum_abl_calculation/	Legitimate Site
9	https://www.thermofisher.com/antibody/primary/target/c-abl	Legitimate Site
10	https://shop.blksport.com/abl	Legitimate Site

Check More URLs

Fig. 6. Working of the proposed model with 10 Legitimate URLs.



Serial	URL	Prediction
1	http://www.posteita.it/Poste.html	Phishing Site
2	http://hongarije-vakantiehuizen.com/cdi-eduu/certificates/includes/alternate/proxy/sign/in/linkedin/SignIn.php	Phishing Site
3	http://payfull.com.mx/plugins/editors-xttd/pagebreak/dbd348abe1c92fcab35213d3357e9575M2ZIN2jYjAeMGVYmE1MzNjYzlhMWM3NzcwOGU1ZjJk=/myaccount/websec_login/	Phishing Site
4	http://cox.com-6159368de39251d7a-login.id-107sbt9cbhsbtd5d80a13c0db1f546757jmq9j5754675733270374.kylelierman.com/IIOysTgNjFrGtHtEAwVo/index.php	Phishing Site
5	http://consultefgtscaxaseg.com/d/	Phishing Site
6	http://36hourdating.com/sitemapo/Sign.htm	Phishing Site
7	http://cantilo.com/wq2o0s/officemi/	Phishing Site
8	http://storepalsa.id/wp-includes/js/mediaelement/request/sbcglobal.htm	Phishing Site
9	http://alfatra.com/night/Yah/T/Y1.html	Phishing Site
10	http://mcts-qatar.com/js/server/index.php	Phishing Site

Check More URLs

Fig. 7. Working of the proposed model with 10 Phishing URLs.

to expand support for multilingual URLs and encoded formats to improve accuracy across diverse domains. We also plan to incorporate URL expansion techniques to enhance detection of shortened or obfuscated URLs. Despite these challenges, the proposed system marks a significant advancement in real-time phishing detection.

Conclusion and future work

This study focuses on extracting diverse features from URLs to detect phishing attacks effectively. The features are derived from three main sources: handcrafted, transformer-based model, and DL architectures. Among these, features extracted using LSTM with multi-attention mechanisms show superior performance to those from handcrafted and transformer-based features. Also, multiple ML algorithms were applied to the classification task to evaluate the effectiveness of the extracted features. The experimental results revealed that the majority of

Reference	Response time (ms)
Dunlop et al. ⁵⁹	4310
Huh et al. ⁶⁰	500
Varshney et al. ⁶¹	1530
Amrutkar et al. ⁴⁹	829
Canali et al. ¹³	3100
Ma et al. ⁷	3580
Rao et al. ³⁷	621
Phish-Jam	480

Table 12. Comparison of our proposed work with existing works.

the ML algorithms achieved an accuracy exceeding 90%, reflecting the effectiveness of selected features. Finally, an ensemble of multiple ML algorithms named a super learner ensemble is used to include the diverse behavior of each classifier for the classification of phishing URLs. This ensemble leverages individual classifiers’ unique strengths and diverse behaviors to improve overall performance. By aggregating these predictions, the super learner ensemble shows significantly better results than traditional ML algorithms. It achieved an accuracy of 98.93%, a precision of 99.15%, and an F1-score of 99.07%. In the future, we aim to integrate diverse lexical and content-based features, including those extracted from source code and third-party services such as DNS records and search engine results. We also plan to explore advanced lexical features and optimize feature selection to retain only the most significant features, further improving phishing detection accuracy and adaptability.

Data availability

The dataset used in this study is publicly accessible at <https://github.com/testmail9440/PhishDump> (Accessed: 2025-01-02).

Received: 6 January 2025; Accepted: 9 May 2025
Published online: 15 May 2025

References

1. PhishLabs. 2023 *Phishing Trends and Intelligence Report* (Accessed 02 January 2025) (2023).
2. Microsoft. 2024 *Global Tech Support Scam Research Report* (Accessed 02 January 2025) (2024).
3. APWG. *APWG Phishing Activity Trends Reports* (Accessed 02 January 2025) (2024).
4. Prasad, A. & Chandra, S. Phiusiil: A diverse security profile empowered phishing url detection framework based on similarity index and incremental learning. *Comput. Secur.* **136**, 103545 (2024).
5. Rao, R. S. & Pais, A. R. Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Comput. Appl.* **31**, 3851–3873 (2019).
6. Sahoo, D., Liu, C. & Hoi, S. C. Malicious url detection using machine learning: A survey. <http://arxiv.org/abs/1701.07179> (2017).
7. Ma, J., Saul, L. K., Savage, S. & Voelker, G. M. Beyond blacklists: Learning to detect malicious web sites from suspicious urls. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* 1245–1254 (2009).
8. Prakash, P., Kumar, M., Kompella, R. R. & Gupta, M. Phishnet: predictive blacklisting to detect phishing attacks. In *2010 Proceedings IEEE INFOCOM* 1–5 (IEEE, 2010).
9. Ma, J., Saul, L. K., Savage, S. & Voelker, G. M. Identifying suspicious urls: An application of large-scale online learning. In *Proceedings of the 26th Annual International Conference on Machine Learning* 681–688 (2009).
10. McGrath, D. K. & Gupta, M. Behind phishing: An examination of phisher modi operandi. *LEET* **8**, 4 (2008).
11. Mourtajji, Y., Bouhorma, M., Alghazzawi, D., Aldabbagh, G. & Alghamdi, A. Hybrid rule-based solution for phishing url detection using convolutional neural network. *Wirel. Commun. Mob. Comput.* **2021**, 8241104 (2021).
12. Kondaiah, C., Pais, A. R. & Rao, R. S. Enhanced malicious traffic detection in encrypted communication using tls features and a multi-class classifier ensemble. *J. Netw. Syst. Manag.* **32**, 76 (2024).
13. Canali, D., Cova, M., Vigna, G. & Kruegel, C. Prophiler: A fast filter for the large-scale detection of malicious web pages. In *Proceedings of the 20th International Conference on World Wide Web* 197–206 (2011).
14. Jain, A. K. & Gupta, B. B. A machine learning based approach for phishing detection using hyperlinks information. *J. Ambient. Intell. Humaniz. Comput.* **10**, 2015–2028 (2019).
15. Xiang, G., Hong, J., Rose, C. P. & Cranor, L. Cantina+ a feature-rich machine learning framework for detecting phishing web sites. *ACM Trans. Inf. Syst. Secur.* **14**, 1–28 (2011).
16. Chiew, K. L., Tan, C. L., Wong, K., Yong, K. S. & Tiong, W. K. A new hybrid ensemble feature selection framework for machine learning-based phishing detection system. *Inf. Sci.* **484**, 153–166 (2019).
17. Chin, T., Xiong, K. & Hu, C. Phishlimiter: A phishing detection and mitigation approach using software-defined networking. *IEEE Access* **6**, 42516–42531 (2018).
18. Zhang, W., Jiang, Q., Chen, L. & Li, C. Two-stage elm for phishing web pages detection using hybrid features. *World Wide Web* **20**, 797–813 (2017).
19. Ding, Y., Luktarhan, N., Li, K. & Slamun, W. A keyword-based combination approach for detecting phishing webpages. *Comput. Secur.* **84**, 256–275 (2019).
20. Lin, Y. et al. Phishpedia: A hybrid deep learning based approach to visually identify phishing webpages. In *30th USENIX Security Symposium (USENIX Security 21)* 3793–3810 (2021).
21. Kondaiah, C., Pais, A. R. & Rao, R. S. An ensemble learning approach for detecting phishing urls in encrypted tls traffic. *Telecommun. Syst.* **1**, 1–17 (2024).
22. Liu, X. & Fu, J. Spwalk: Similar property oriented feature learning for phishing detection. *Ieee Access* **8**, 87031–87045 (2020).
23. Rao, R. S., Pais, A. R. & Anand, P. A heuristic technique to detect phishing websites using twsvm classifier. *Neural Comput. Appl.* **33**, 5733–5752 (2021).
24. Tan, C. L. et al. A graph-theoretic approach for the detection of phishing webpages. *Comput. Secur.* **95**, 101793 (2020).

25. Geng, G.-G., Lee, X.-D. & Zhang, Y.-M. Combating phishing attacks via brand identity and authorization features. *Secur. Commun. Netw.* **8**, 888–898 (2015).
26. Alani, M. M. & Tawfik, H. Phishnot: A cloud-based machine-learning approach to phishing url detection. *Comput. Netw.* **218**, 109407 (2022).
27. Oest, A. et al. *phishitime*: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists. In *29th USENIX Security Symposium (USENIX Security 20)* 379–396 (2020).
28. Rao, R. S., Vaishnavi, T. & Pais, A. R. Catchphish: Detection of phishing websites by inspecting urls. *J. Ambient. Intell. Humaniz. Comput.* **11**, 813–825 (2020).
29. Le, H., Pham, Q., Sahoo, D. & Hoi, S. C. Urlnet: Learning a url representation with deep learning for malicious url detection. <http://arxiv.org/abs/1802.03162> (2018).
30. Maneriker, P. et al. Urltran: Improving phishing url detection using transformers. In *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)* 197–204 (IEEE, 2021).
31. Kumar, M., Kondaiah, C., Pais, A. R. & Rao, R. S. Machine learning models for phishing detection from tls traffic. *Clust. Comput.* **26**, 3263–3277 (2023).
32. Kashyap, H., Pais, A. R. & Kondaiah, C. Machine learning-based malware detection and classification in encrypted tls traffic. In *International Conference on Security, Privacy and Data Analytics* 247–262 (Springer, 2022).
33. Mamun, M. S. I., Rathore, M. A., Lashkari, A. H., Stakhanova, N. & Ghorbani, A. A. Detecting malicious urls using lexical analysis. In *Network and System Security: 10th International Conference, NSS 2016, Taipei, Taiwan, September 28–30, 2016, Proceedings 10* 467–482 (Springer, 2016).
34. Sahingoz, O. K., Buber, E., Demir, O. & Diri, B. Machine learning based phishing detection from urls. *Expert Syst. Appl.* **117**, 345–357 (2019).
35. Yuan, H., Yang, Z., Chen, X., Li, Y. & Liu, W. Url2vec: Url modeling with character embeddings for fast and accurate phishing website detection. In *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)* 265–272 (IEEE, 2018).
36. McCormick, C. Word2vec Tutorial-the Skip-Gram Model. <http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model> (2016).
37. Rao, R. S., Vaishnavi, T. & Pais, A. R. Phishdump: A multi-model ensemble based technique for the detection of phishing sites in mobile devices. *Pervas. Mob. Comput.* **60**, 101084 (2019).
38. Aljofey, A., Jiang, Q., Qu, Q., Huang, M. & Niyigena, J.-P. An effective phishing detection model based on character level convolutional neural network from url. *Electronics* **9**, 1514 (2020).
39. Kenton, J. & Toutanova, L. K. Bert: Pre-training of deep bidirectional transformers for language understanding. *Proc. NAACL-HLT* **1**, 2 (2019).
40. Li, Y., Yang, Z., Chen, X., Yuan, H. & Liu, W. A stacking model using url and html features for phishing webpage detection. *Futur. Gener. Comput. Syst.* **94**, 27–39 (2019).
41. Das Gupta, S., Shahriar, K. T., Alqahtani, H., Alsaman, D. & Sarker, I. H. Modeling hybrid feature-based phishing websites detection using machine learning techniques. *Ann. Data Sci.* **11**, 217–242 (2024).
42. Selvaganapathy, S., Nivaashini, M. & Natarajan, H. Deep belief network based detection and categorization of malicious urls. *Inf. Secur. J. Glob. Perspect.* **27**, 145–161 (2018).
43. Bahnsen, A. C., Bohorquez, E. C., Villegas, S., Vargas, J. & González, F. A. Classifying phishing urls using recurrent neural networks. In *2017 APWG Symposium on Electronic Crime Research (eCrime)* 1–8 (IEEE, 2017).
44. Zhao, J., Wang, N., Ma, Q. & Cheng, Z. Classifying malicious urls using gated recurrent neural networks. In *Innovative Mobile and Internet Services in Ubiquitous Computing: Proceedings of the 12th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS-2018)* 385–394 (Springer, 2019).
45. Han, W., Wang, Y., Cao, Y., Zhou, J. & Wang, L. Anti-phishing by smart mobile device. In *2007 IFIP International Conference on Network and Parallel Computing Workshops (NPC 2007)* 295–302 (IEEE, 2007).
46. Wu, L., Du, X. & Wu, J. Effective defense schemes for phishing attacks on mobile computing platforms. *IEEE Trans. Veh. Technol.* **65**, 6678–6691 (2015).
47. Orunsolu, A. A. et al. A lightweight anti-phishing technique for mobile phone. *Acta Inform. Pragensia* **6**, 114–123 (2017).
48. Hou, J. & Yang, Q. Defense against mobile phishing attack. *Computer Security Course Project*. <http://www.personal.umich.edu/yanqi/pivot/mobile-phishing-defense.pdf> (2012).
49. Amrutkar, C., Kim, Y. S. & Traynor, P. Detecting mobile malicious webpages in real time. *IEEE Trans. Mob. Comput.* **16**, 2184–2197 (2016).
50. Chorghhe, S. P. & Shekokar, N. A solution to detect phishing in android devices. In *Information Systems Security: 12th International Conference, ICISS 2016, Jaipur, India, December 16–20, 2016, Proceedings 12* 461–470 (Springer, 2016).
51. Ndiwile, J. D., Kadobayashi, Y. & Fall, D. Unphishme: Phishing attack detection by deceptive login simulation through an android mobile app. In *2017 12th Asia Joint Conference on Information Security (AsiaJIS)* 38–47 (IEEE, 2017).
52. Haynes, K., Shirazi, H. & Ray, I. Lightweight url-based phishing detection using natural language processing transformers for mobile devices. *Procedia Comput. Sci.* **191**, 127–134 (2021).
53. Jain, A. K., Debnath, N. & Jain, A. K. Apuml: An efficient approach to detect mobile phishing webpages using machine learning. *Wirel. Pers. Commun.* **125**, 3227–3248 (2022).
54. Dhanavanthini, P. & Chakkravarthy, S. S. Phish-armour: Phishing detection using deep recurrent neural networks. *Soft Comput.* **1**, 1–13 (2023).
55. Gupta, B. B., Gaurav, A. & Chui, K. T. Securing smartphone from mobile phishing attacks using googlenet model. In *2024 IEEE International Symposium on Consumer Technology (ISCT)* 522–527 (IEEE, 2024).
56. Ponsam, J. G. et al. Url shield: Protecting users from phishing attacks using flask and ml. In *2024 3rd International Conference for Innovation in Technology (INOCON)* 1–5 (IEEE, 2024).
57. Vaswani, A. et al. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, 1 (2017).
58. Patgiri, R., Katari, H., Kumar, R. & Sharma, D. Empirical study on malicious url detection using machine learning. In *Distributed Computing and Internet Technology: 15th International Conference, ICDCIT 2019, Bhubaneswar, India, January 10–13, 2019, Proceedings 15* 380–388 (Springer, 2019).
59. Dunlop, M., Groat, S. & Shelly, D. Goldphish: Using images for content-based phishing analysis. *2010 Fifth International Conference on Internet Monitoring And Protection* 123–128 (2010).
60. Huh, J. H. & Kim, H. Phishing detection with popular search engines: Simple and effective. In *Foundations and Practice of Security: 4th Canada-France MITACS Workshop, FPS 2011, Paris, France, May 12–13, 2011, Revised Selected Papers 4* 194–207 (Springer, 2012).
61. Varshney, G., Misra, M. & Atrey, P. K. A phish detector using lightweight search features. *Comput. Secur.* **62**, 213–228 (2016).

Author contributions

All authors contributed equally to this work.

Funding

This study was supported by research fund from Chosun University, 2024.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to B.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025